

DASH: Divergence-Adaptive Supervision Horizons for On-Policy Self-Distillation of Reasoning Models

ZhiYan Hou^{*1,2,3}, Xinyu Tang^{*2,3}, Hongyan An¹, Jianjin Zhang^{2,3},
Weizhen Wang^{2,3}, Yunyun Han^{2,3}, Gengsheng Li¹, Xiangzhao Hao¹,
Haiyun Guo^{1,4}, Wenbin Hu^{6,†}, Jinqiao Wang^{1,4,5}, Yafeng Deng^{2,3,†}

¹Institute of Automation, Chinese Academy of Sciences

²EverMind

³Shanda Group

⁴University of Chinese Academy of Sciences

⁵Wuhan AI Research

⁶Wuhan University

[†]Corresponding authors.

Abstract

Reinforcement learning with verifiable rewards (RLVR) improves the reasoning capabilities of large language models using automatically verifiable outcome signals, but these signals are typically sparse and at the sequence-level. On-policy self-distillation (OPSD) mitigates this sparsity by querying a privileged teacher at student-visited prefixes and providing dense token-level distributional supervision. Although this dense supervision alleviates signal sparsity, we find that standard OPSD still underexploits the temporal structure of the rollout. It assigns every local divergence the same coefficient, regardless of its position or the divergence sequence in which it occurs. In on-policy autoregressive generation, the same divergence magnitude can follow different discrepancy histories, reflecting different evolutions of the mismatch between the teacher and student. Since the local scalar alone cannot distinguish these temporal contexts, standard OPSD cannot adapt its token-level weights to the realized discrepancy sequence. To address this limitation, we propose *Divergence-Adaptive Supervision Horizons* (DASH). DASH maps the gap between each local distillation signal and the sequence-level mean to an adaptive propagation gate and then uses these gates to control backward multi-step aggregation. By doing so, DASH adjusts token-level supervision weights according to how local divergences evolve during generation. Experiments on three mathematical reasoning benchmarks across three model scales show that DASH improves over our matched vanilla OPSD reruns on every benchmark at all three scales. DASH reuses the teacher and student distributions that OPSD already computes, so the gains require no additional teacher or student forward pass.

Code — <https://github.com/DBtxy/DASH-OPSD>

1 Introduction

Reinforcement learning with verifiable rewards (RLVR) has become an important post-training paradigm for improving the mathematical reasoning and code generation capabilities of large language models (Lambert et al. 2024; Shao et al.

2024; Guo et al. 2025). It optimizes models using automatically verifiable signals, such as answer correctness and program execution results (Cobbe et al. 2021; Hendrycks et al. 2021; Chen et al. 2021), avoiding costly human preference annotations (Ouyang et al. 2022; Rafailov et al. 2023) and scales effectively (Yu et al. 2025). However, standard RLVR typically applies the same sparse sequence-level outcome reward to all token-level decisions in a response, making it difficult to distinguish their individual contributions to the final outcome (Uesato et al. 2022; Lightman et al. 2024). This temporal credit assignment challenge limits both sample efficiency and optimization stability in long-horizon reasoning (Sutton 1988; Kazemnejad et al. 2024; Parthasarathi et al. 2025).

To mitigate this coarse credit assignment, recent work has introduced on-policy distillation (OPD) and on-policy self-distillation (OPSD) (Agarwal et al. 2024; Gu et al. 2024; Zhao et al. 2026), which provide dense token-level supervision on student-generated trajectories (Hinton, Vinyals, and Dean 2015; Zelikman et al. 2022). In OPSD, the student generates a response, while the same model, conditioned on a reference answer or other privileged information (Vapnik and Vashist 2009; Lopez-Paz et al. 2016), acts as the teacher and provides distributional supervision at the student-visited prefixes.

However, dense token-level supervision primarily alleviates signal sparsity, while leaving the temporal structure of the rollout underexploited. This is because vanilla OPSD aggregates position-wise distributional discrepancies using uniform coefficients that are independent of both token position and the temporal discrepancy profile along the rollout. But in on-policy autoregressive generation (Ross, Gordon, and Bagnell 2011; Bengio et al. 2015), the same local discrepancy value can arise after different discrepancy histories that reflect how the mismatch between teacher and student has evolved over student-visited prefixes. The local scalar alone cannot distinguish these temporal contexts, yet vanilla OPSD assigns the same explicit aggregation coefficient in every case. Consequently, vanilla OPSD cannot adapt token-

^{*}These authors contributed equally.

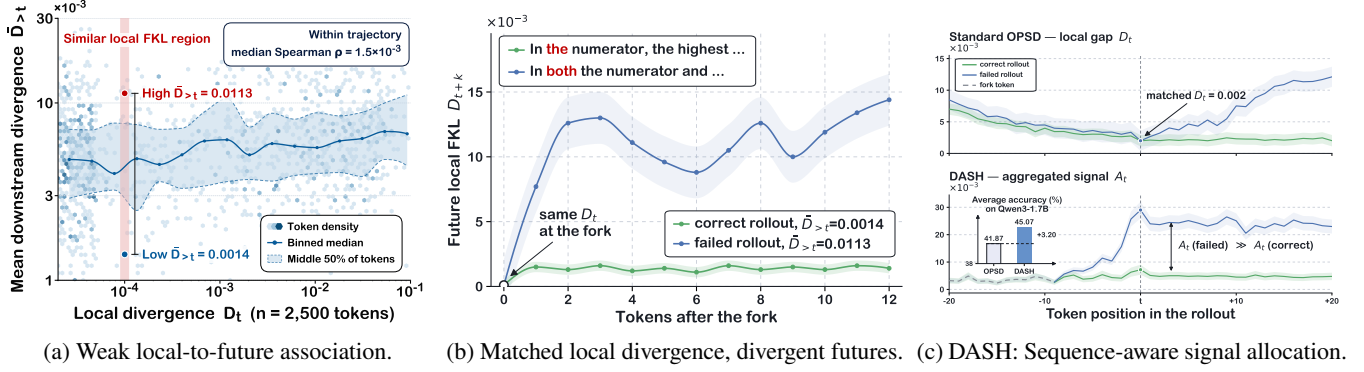


Figure 1: **Motivation and mechanism of DASH.** (a) The current local divergence D_t is weakly associated with the mean future divergence $\bar{D}_{>t}$ within student-generated trajectories. The plot visualizes 2,500 of the 30k analyzed tokens. (b) Two rollouts matched by local divergence at a reasoning fork subsequently exhibit sharply different divergence profiles and terminal outcomes. (c) Vanilla OPSD aggregates the matched local divergences with the same coefficient, regardless of temporal discrepancy profiles. DASH instead uses the realized discrepancy sequence to produce path-dependent aggregated supervision signals.

level distillation weights to the temporal evolution of the discrepancy sequence.

To make coefficient allocation sensitive to the temporal discrepancy profile, we propose *Divergence-Adaptive Supervision Horizons* (DASH), a divergence-adaptive multi-step aggregation method for OPSD. At each position, DASH converts the gap between the local distillation signal and its sequence-level mean into an adaptive propagation gate λ_t . These gates control a backward multi-step recursion, yielding a dynamic coefficient for each local loss that is determined by its preceding gate path. DASH thus replaces the uniform coefficient profile of vanilla OPSD with a path-dependent sequence-level objective whose effective supervision horizon adapts to the realized discrepancy sequence.

Our contributions are threefold. **Firstly**, we identify a temporal coefficient allocation gap in vanilla OPSD: assigning the same weight to every local divergence makes the objective invariant to different temporal arrangements of the same divergence values, preventing supervision weights from adapting to temporal structure (Section 4.1). **Secondly**, we propose *Divergence-Adaptive Supervision Horizons* (DASH), which uses the gap between each local divergence and the sequence-level mean to adapt the effective supervision horizon and produces temporally conditioned token-level weights through multi-step aggregation, enabling dense distillation supervision to exploit temporal structure (Section 4.2). **Thirdly**, with negligible additional training overhead, DASH obtains the highest overall scores among the compared results across three mathematical reasoning benchmarks and three model scales (Table 1), supported by comprehensive ablation studies (Sections 5.3 and 5.4).

2 Related Work

2.1 Reinforcement Learning with Verifiable Rewards

Reinforcement learning with verifiable rewards (RLVR) improves language-model reasoning using automatically checked signals, such as answer matching, program ex-

cution, and unit tests (Lambert et al. 2024; Cobbe et al. 2021; Chen et al. 2021). DeepSeekMath introduced Group Relative Policy Optimization (GRPO), while DeepSeek-R1 demonstrated the effectiveness of large-scale RLVR for mathematical and coding reasoning (Shao et al. 2024; Guo et al. 2025). Standard RLVR nevertheless relies primarily on sparse sequence-level rewards (Schulman et al. 2017; Yu et al. 2025; Yue et al. 2025), providing limited direct supervision for intermediate decisions in long reasoning trajectories (Wei et al. 2022; Uesato et al. 2022). Process supervision introduces step-level feedback through process reward models (Lightman et al. 2024; Uesato et al. 2022; Wang et al. 2024; Setlur et al. 2025), whereas temporal credit-assignment methods such as GRPO- λ redistribute outcome-level signals across sampled decisions using eligibility traces and critic-free temporal-difference estimates (Parthasarathi et al. 2025; Sutton 1988; Schulman et al. 2016; Kazemnejad et al. 2024). These approaches address learning from sparse reward signals; Our work investigates how dense, differentiable token-level supervision in OPSD should be allocated across a rollout.

2.2 On-Policy Self-Distillation

On-policy self-distillation (OPSD) trains a model on its own rollouts by using the same model under different contexts as an unprivileged student and a privileged teacher (Agarwal et al. 2024; Gu et al. 2024; Zelikman et al. 2022). The teacher conditions on reference solutions or other privileged information and provides token-level distributional supervision at student-visited prefixes (Zhao et al. 2026; Vapnik and Vashist 2009; Lopez-Paz et al. 2016; Pinto et al. 2018; Weihs et al. 2021). Recent OPSD methods refine either privileged supervision or token weighting (Jin et al. 2026; Li et al. 2026; Xie et al. 2026; Lin, Chen, and Zhang 2026; Shen et al. 2026; Liu et al. 2026b). AVSD combines cross-view consensus with view-specific signals from multiple privileged teacher contexts (Nguyen et al. 2026), while PW-OPSD applies fixed position-dependent weights motivated by vari-

ation in teacher-token reliability (Liu et al. 2026a). These methods improve the supervision source or impose predefined positional weighting. DASH instead focuses on how local distillation signals are aggregated across the rollout. It uses sequence-relative divergence gaps to construct adaptive propagation gates and temporally conditioned token-level coefficients through multi-step aggregation, allowing dense supervision to exploit the temporal evolution of local discrepancies.

3 Preliminaries

Let π_θ denote the student policy. Given a problem x , the student samples a response $\mathbf{y} = (y_1, \dots, y_T) \sim P_\theta(\cdot | x)$. We denote the student-generated prefix at position t by $s_t = (x, y_{<t})$.

On-policy self-distillation (OPSD) (Agarwal et al. 2024; Zhao et al. 2026) uses a single model as both the student and a privileged teacher under different conditioning contexts. The student generates the rollout from the problem x , while the teacher additionally conditions on privileged information z , such as a reference solution. At each student-visited prefix s_t , their distributions are

$$\pi_t^S = \pi_\theta(\cdot | s_t), \quad \pi_t^T = \text{sg}[\pi_{\bar{\theta}}(\cdot | s_t, z)], \quad (1)$$

where $\bar{\theta}$ denotes the teacher parameters and $\text{sg}[\cdot]$ blocks gradients through the teacher target. Querying the privileged teacher along the student rollout provides dense token-level distributional supervision (Agarwal et al. 2024; Liu et al. 2026b).

Our main setting uses the forward KL divergence from the privileged teacher to the student as the local distillation loss (Hinton, Vinyals, and Dean 2015; Kim and Rush 2016):

$$d_t = D_{\text{KL}}(\pi_t^T \| \pi_t^S) = \sum_{v \in \mathcal{A}} \pi_t^T(v) \log \frac{\pi_t^T(v)}{\pi_t^S(v)}. \quad (2)$$

For a sampled response $\mathbf{y}$, vanilla OPSD uniformly aggregates the local distillation losses:

$$\mathcal{L}_{\text{OPSD}}(\theta; \mathbf{y}) = \frac{1}{T} \sum_{t=1}^T d_t. \quad (3)$$

Vanilla OPSD treats the sampled response as fixed during optimization and differentiates Eq. (3) only through the student distributions (Agarwal et al. 2024; Liu et al. 2026b).

4 Method

Vanilla OPSD assigns the same explicit coefficient to every local distillation loss, regardless of its position or the surrounding evolution of discrepancies along the rollout. We first characterize this limitation and use the exact gradient of the corresponding expected on-policy objective as a structural reference. Beyond the direct distillation term retained by standard training, the exact gradient contains a trajectory term whose coefficient depends on subsequent divergences.

Motivated by this order-dependent structure, we introduce DASH, which uses sequence-relative divergence gaps, adaptive propagation gates, and backward multi-step aggregation to construct temporally conditioned coefficients for direct distillation gradients. Figure 2 provides an overview.

4.1 Coefficient Structure of Vanilla OPSD

For a sampled rollout $\mathbf{y}$, vanilla OPSD retains the detached gradient

$$\nabla_\theta \mathcal{L}_{\text{OPSD}}(\theta; \mathbf{y}) = \frac{1}{T} \sum_{t=1}^T \nabla_\theta^{\text{loc}} d_t. \quad (4)$$

Thus, every local gradient has the same explicit coefficient $1/T$, independent of the ordered discrepancy profile $\mathbf{d} = (d_1, \dots, d_T)$. This does not make the local gradients sequence-independent: each $\nabla_\theta^{\text{loc}} d_t$ is still evaluated at its corresponding student-generated prefix s_t . Only the explicit coefficient profile is independent of how the local discrepancies evolve along the rollout.

To isolate coefficient structure, consider the following fixed-horizon surrogate. Let H be a deterministic truncation horizon; after EOS, the trajectory enters a parameter-independent absorbing state and $d_t = 0$ thereafter.

$$\mathcal{J}_H(\theta) = \mathbb{E}_{(x,z) \sim \mathcal{D}} \mathbb{E}_{\mathbf{y} \sim P_\theta(\cdot | x)} \left[\frac{1}{H} \sum_{t=1}^H d_t \right]. \quad (5)$$

Proposition 1 (Fixed-Horizon Gradient Decomposition). *Under the fixed-horizon construction above, the gradient of Eq. (5) is*

$$\begin{aligned} \nabla_\theta \mathcal{J}_H(\theta) = \mathbb{E} \left[\frac{1}{H} \sum_{t=1}^H \nabla_\theta^{\text{loc}} d_t \right. \\ \left. + \frac{1}{H} \sum_{u=1}^H G_{u+1}^D \nabla_\theta \log \pi_\theta(y_u | s_u) \right], \end{aligned} \quad (6)$$

where $G_{u+1}^D = \sum_{t=u+1}^H d_t$ is the future divergence-to-go after position u .

Proof. Let $D_{\text{tot}}(\mathbf{y}; \theta) = \sum_{t=1}^H d_t$. Applying the log-derivative identity (Williams 1992; Sutton et al. 2000) to the trajectory expectation gives

$$\begin{aligned} \nabla_\theta \mathcal{J}_H(\theta) = \frac{1}{H} \mathbb{E} \left[\sum_{t=1}^H \nabla_\theta^{\text{loc}} d_t \right. \\ \left. + D_{\text{tot}} \sum_{u=1}^H \nabla_\theta \log \pi_\theta(y_u | s_u) \right]. \end{aligned} \quad (7)$$

For $t \leq u$, the value of d_t is determined before y_u is sampled and is therefore measurable with respect to s_u . Using the zero conditional mean of the score function,

$$\mathbb{E}_{y_u \sim \pi_\theta(\cdot | s_u)} [d_t \nabla_\theta \log \pi_\theta(y_u | s_u) | s_u] = 0, \quad t \leq u. \quad (8)$$

Hence, the score term at position u retains only losses at positions $t > u$, whose sum is $G_{u+1}^D = \sum_{t=u+1}^H d_t$. Substituting this result into Eq. (7) yields Eq. (6). $\square$

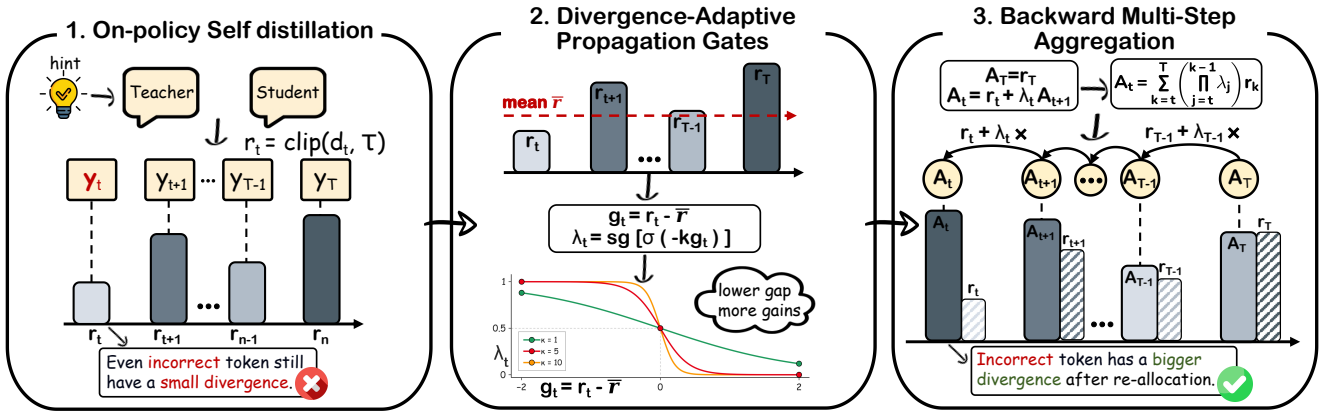


Figure 2: Overview of DASH. A privileged teacher evaluates a student-generated rollout to produce local distillation signals. DASH converts their sequence-relative gaps into adaptive propagation gates and applies backward multi-step aggregation to construct temporally conditioned token-level weights and adaptive supervision horizons.

The first term is the direct distillation gradient retained by vanilla OPSD. The second is a trajectory score-function term with future-divergence coefficients, reflecting changes in future visited prefixes when the sampling path is differentiated (Ross, Gordon, and Bagnell 2011; Bengio et al. 2015). This fixed-horizon decomposition provides only structural motivation: DASH does not estimate G_{u+1}^D , introduce score-function gradients, or perform future-to-past credit assignment.

4.2 Divergence-Adaptive Supervision Horizons

The fixed-horizon decomposition highlights a structural contrast: order dependence appears in the trajectory coefficients, whereas the direct distillation term retained by vanilla OPSD uses constant explicit coefficients. Motivated by this contrast, DASH makes the coefficients of direct local distillation losses depend on the realized discrepancy sequence through adaptive supervision horizons.

Using the propagation gates, DASH applies the backward recursion (Sutton 1988; Schulman et al. 2016)

$$A_T = r_T, \quad A_t = r_t + \lambda_t A_{t+1}, \quad t = T-1, \dots, 1, \quad (9)$$

and minimizes

$$\mathcal{L}_{\text{DASH}} = \frac{1}{T} \sum_{t=1}^T A_t. \quad (10)$$

Here, λ_t controls how strongly later local signals propagate across the boundary between positions t and $t+1$ (Hochreiter and Schmidhuber 1997; Cho et al. 2014). In particular, for $k > t$, the contribution of r_k to A_t is scaled by $\prod_{j=t}^{k-1} \lambda_j$. Summing these contributions over all aggregation starting positions gives the weighted form

$$\mathcal{L}_{\text{DASH}} = \frac{1}{T} \sum_{k=1}^T c_k r_k, \quad c_1 = 1, \quad c_k = 1 + \lambda_{k-1} c_{k-1}. \quad (11)$$

Algorithm 1: DASH aggregation for one student rollout. Gradients flow only through the local signals r_t .

Input: Local KL contributions $\{\ell_{t,v}\}$, clipping threshold τ , and propagation sensitivity κ

```

1: for  $t = 1, \dots, T$  do
2:    $r_t \leftarrow \sum_{v \in \mathcal{A}} \min(\ell_{t,v}, \tau)$ 
3: end for
4:  $\bar{r} \leftarrow \frac{1}{T} \sum_{t=1}^T r_t$ 
5: for  $t = 1, \dots, T-1$  do
6:    $\lambda_t \leftarrow \text{sg}[\sigma(-\kappa(r_t - \bar{r}))]$ 
7: end for
8:  $A_T \leftarrow r_T$ 
9: for  $t = T-1, T-2, \dots, 1$  do
10:   $A_t \leftarrow r_t + \lambda_t A_{t+1}$ 
11: end for
12: return  $\frac{1}{T} \sum_{t=1}^T A_t$ 

```

Thus, c_k aggregates the contributions to r_k from all starting positions up to k , weighted by the intervening gates. Although its recursion involves $\lambda_1, \dots, \lambda_{k-1}$, each gate is computed relative to the sequence-level mean $\bar{r}$. Consequently, c_k is conditioned on the realized discrepancy sequence.

Let $\ell_{t,v}$ denote the vocabulary-level summand in Eq. (2). Algorithm 1 summarizes the additional DASH computation after the local KL contributions have been obtained along a rollout.

5 Experiments

5.1 Experimental Setup

Models and Datasets. We conduct experiments with the Qwen3-1.7B, Qwen3-4B, and Qwen3-8B Instruct models (Yang et al. 2025). Our reruns use OpenThoughts-Math-30K (Guha et al. 2025), which contains 29,434 mathematical reasoning examples, each with a problem, reference solution, and final answer. In every on-policy distillation run, the student receives only the problem and generates its own rollout. For OPSD, EOPD, PW-OPSD, and DASH, the matched privileged teacher additionally receives the refer-

ence solution, including the final answer; AVSD retains its method-defining multi-view privileged contexts. We evaluate on AIME 2024 (Mathematical Association of America 2024), AIME 2025 (Mathematical Association of America 2025), and HMMT February 2025 (Harvard-MIT Mathematics Tournament 2025), each containing 30 problems, and determine correctness using the official answer keys. For each benchmark, Avg@12 averages the binary correctness of 12 independently sampled responses per problem and then averages over all problems. We additionally report the unweighted mean across the three benchmarks.

Baselines. We compare DASH with seven baselines. *Base* denotes the original model without additional training. *SFT* performs supervised fine-tuning on the reference solutions (Ouyang et al. 2022). *GRPO* optimizes verifiable rewards based on final-answer correctness (Shao et al. 2024). *OPSD* uniformly aggregates token-level distillation losses along student-generated trajectories (Zhao et al. 2026). *EOPD* adapts token-level distillation using teacher entropy (Jin et al. 2026). *AVSD* combines supervision from multiple privileged teacher views (Nguyen et al. 2026). *PW-OPSD* applies predefined position-dependent weights to token-level distillation losses (Liu et al. 2026a). Table 1 combines externally reported reference results with our reruns. Base, SFT, and GRPO are taken from Zhao et al. (Zhao et al. 2026) and marked with †. We rerun OPSD, EOPD, AVSD, PW-OPSD, and DASH. OPSD, EOPD, PW-OPSD, and DASH use the same privileged-OPSD backbone while retaining their method-specific objectives or weighting rules; AVSD retains its original multi-view teacher construction. Exact method instantiations, run counts, and result provenance are provided in Appendix B.

Implementation Details. All methods rerun by us start from the corresponding Qwen3 Instruct checkpoint, train for the full 200-step budget, and use the same checkpoint-selection and benchmark-evaluation procedures. We save checkpoints every 20 steps and, for each method, model scale, and training seed, select the checkpoint within 200 steps that maximizes the unweighted average across the three benchmarks; all three benchmark scores are taken from that single checkpoint. We refer to this as best-within-200-step reporting rather than held-out validation selection.

Following the short-rollout OPSD setting studied by Zhao et al. (Zhao et al. 2026), the matched privileged-OPSD runs use a maximum student completion length of 1,024 tokens. We additionally evaluate maximum lengths of 2,048 and 4,096 tokens; because they yield comparable performance, we use 1,024 tokens for computational efficiency. Unless otherwise specified, these runs use LoRA (Hu et al. 2022) with rank 64 and scaling factor 128, a learning rate of 5×10^{-6} , and a global batch size of 64. Student rollouts are sampled with temperature 1.1, top- p 0.95, and top- k 20. DASH uses full-vocabulary forward KL as its local distillation loss, caps each vocabulary-level divergence contribution at $\tau = 0.05$, and sets $\kappa = 5$. For Avg@12 evaluation, we independently sample 12 responses per problem in thinking mode using temperature 1.0, top- p 1.0, and a maximum of 38,912 newly generated tokens. OPSD, DASH, and every DASH ablation

configuration use training seeds $s \in \{0, 1, 2, 3\}$ and report four-seed means. Complete provenance, method-specific settings, checkpoint selection, prompts, run counts, and seed statistics are provided in Appendices B and C.

5.2 Main Results

Table 1 compares DASH with standard training baselines and recent on-policy distillation methods across three model scales and three mathematical reasoning benchmarks.

Overall performance. DASH obtains the highest score among the compared results in all nine benchmark-model settings and the highest macro-average at each model scale. Under the matched rerun protocol, DASH raises the four-seed OPSD average from 41.87 to 45.07 on Qwen3-1.7B, from 63.60 to 65.00 on Qwen3-4B, and from 64.80 to 66.40 on Qwen3-8B. The corresponding gains are 3.20, 1.40, and 1.60 points. DASH also improves over the matched OPSD rerun on every benchmark at every model scale, indicating that its gains are not confined to a particular model capacity or dataset.

Comparison with recent distillation methods. Among the displayed comparisons, PW-OPSD provides the highest competing macro-average at the 1.7B and 8B scales, whereas OPSD provides the highest at the 4B scale. DASH exceeds the compared PW-OPSD result by 1.64 points on Qwen3-1.7B and by 1.17 points on Qwen3-8B, and it exceeds the matched OPSD result by 1.40 points on Qwen3-4B. Thus, DASH attains the best displayed scores among the compared methods, without implying a statistical significance test against the single-run baselines.

5.3 Component Ablation

Having established the overall effectiveness of DASH, we next examine which aspects of its adaptive coefficient allocation account for the observed gains. Unless otherwise specified, all experiments in this subsection use Qwen3-1.7B, the same four training seeds, and the same training, checkpoint-selection, and evaluation protocol as the matched OPSD comparison. We investigate three possible sources of improvement: whether sequence-adaptive propagation is more effective than fixed multi-step aggregation, whether the direction of the gap-to-gate mapping matters, and whether the gains can be explained by the increase in average coefficient scale.

Adaptive versus fixed propagation. We first examine whether adapting the propagation gates to the discrepancy sequence provides benefits beyond multi-step aggregation alone. We replace the token-wise gates λ_t with a fixed coefficient λ shared across all positions and rollouts. These variants retain the same backward aggregation structure as DASH but remove its dependence on the realized discrepancy sequence. We evaluate $\lambda \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$, with $\lambda = 0$ recovering vanilla OPSD.

As shown in Figure 3a, every tested fixed coefficient improves over vanilla OPSD, and the best setting, $\lambda = 0.1$, raises the macro-average from 41.87 to 43.63. This indicates that extending supervision beyond independent local losses

Method	Qwen3-1.7B				Qwen3-4B				Qwen3-8B			
	AIME 2024	AIME 2025	HMMT 2025	Average	AIME 2024	AIME 2025	HMMT 2025	Average	AIME 2024	AIME 2025	HMMT 2025	Average
Base [†]	51.50	36.70	23.10	37.10	74.90	66.40	42.20	61.17	75.80	65.60	43.90	61.77
SFT [†]	48.40	36.30	22.70	35.80	70.20	62.30	43.40	58.63	72.30	64.20	42.90	59.80
GRPO [†]	51.10	38.30	23.70	37.70	75.60	68.10	44.40	62.70	76.40	68.90	46.70	64.00
OPSD	55.60	40.80	29.20	41.87	76.40	68.30	46.10	63.60	77.80	70.80	45.80	64.80
EOPD	51.90	38.10	26.40	38.80	73.90	64.70	41.90	60.17	77.80	70.80	46.40	65.00
AVSD	55.30	37.50	24.70	39.17	76.20	68.30	44.20	62.90	75.40	69.60	47.10	64.03
PW-OPSD	57.80	42.20	30.30	43.43	76.20	67.78	43.33	62.44	77.80	70.80	47.10	65.23
DASH (Ours)	58.30	45.80	31.10	45.07	77.20	71.10	46.70	65.00	78.90	71.40	48.90	66.40

Table 1: Main results across three model scales. Scores are Avg@12 and Average is the unweighted mean across benchmarks. [†] denotes results reported by Zhao et al. (Zhao et al. 2026); all other entries are our reruns. OPSD and DASH report four-seed means. In each column, the top-three distinct compared results are highlighted: **1st**, **2nd**, and **3rd**; tied scores share the same rank.

is beneficial even with a fixed effective horizon. DASH further improves the macro-average to 45.07, outperforming the best fixed setting by 1.44 points. Thus, fixed multi-step aggregation accounts for part of the gain, while adapting the propagation gates to the realized discrepancy sequence provides an additional improvement.

The fixed- λ comparison shows that adaptive propagation is beneficial, but it does not establish how the sequence-relative gap should control the gate. Let $g_t = r_t - \bar{r}$. DASH and Inverse-gap use the same gap, sensitivity, and aggregation recursion, differing only in the sign inside the gate function:

$$\lambda_t^{\text{DASH}} = \text{sg}[\sigma(-\kappa g_t)], \quad \lambda_t^{\text{Inv}} = \text{sg}[\sigma(+\kappa g_t)]. \quad (12)$$

Since λ_t controls whether subsequent local signals cross the boundary after position t , DASH assigns a larger gate when $g_t < 0$ and a smaller gate when $g_t > 0$. It therefore produces a longer effective supervision horizon after a below-average local signal and a shorter horizon after an above-average one. Inverse-gap reverses exactly this behavior. This comparison directly tests our design hypothesis that propagation should remain more open after relatively small discrepancies and become more restricted after unusually large ones.

As shown in Figure 3a, Inverse-gap reaches a macro-average of 42.10. It is only 0.23 points above vanilla OPSD, 1.53 points below the best fixed coefficient, and 2.97 points below DASH. Thus, the gain does not arise from introducing an arbitrary gap-conditioned gate. The result supports the sign used by DASH among the two tested adaptive mappings.

Coefficient allocation versus average scale. Finally, we examine whether DASH benefits from its relative coefficient profile or merely from stronger overall supervision. In the weighted objective $\mathcal{L}_{\text{DASH}} = T^{-1} \sum_k c_k r_k$, the recursive aggregation produces $c_k \geq 1$. DASH therefore changes both the relative allocation of token-level supervision and the average coefficient scale. This introduces an alternative explanation: uniformly increasing all coefficients might reproduce the improvement without using discrepancy-conditioned allocation.

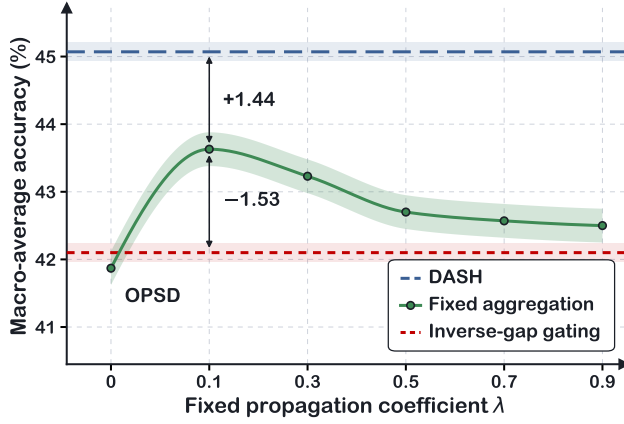
We disentangle relative coefficient allocation from average coefficient scale using the 2×2 factorial comparison in Figure 3b. Normalized DASH preserves the relative coefficient profile at the OPSD scale, while scale-matched OPSD applies uniform coefficients at the DASH scale. Dynamic allocation improves performance by 2.40 points at the OPSD scale and 2.50 points at the DASH scale. In contrast, increasing the average scale contributes only 0.70 points under uniform allocation and 0.80 points under dynamic allocation. The consistent advantage at both scales shows that the improvement primarily comes from DASH’s discrepancy-conditioned coefficient allocation, with the increased scale providing only a smaller complementary benefit.

Overall, the component ablations show that the gains of DASH cannot be reproduced by fixed multi-step aggregation, a reversed adaptive mapping, or uniform coefficient scaling. Its effectiveness primarily arises from adapting the relative coefficient allocation to the discrepancy sequence using the proposed gap-to-gate direction.

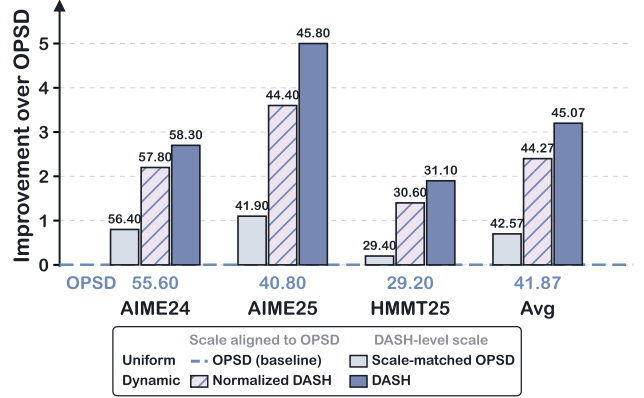
5.4 Design Choices and Sensitivity

We examine three design choices that govern how DASH constructs and propagates token-level supervision. The distillation divergence and vocabulary support determine the local signal used for both direct distillation and gate construction, while the propagation sensitivity controls how this signal shapes the adaptive supervision horizon. These analyses assess the robustness of DASH to its main configuration choices and support the settings used in the main experiments. Unless otherwise specified, all experiments use Qwen3-1.7B, the same four training seeds, and the same training, checkpoint-selection, and evaluation protocol as the matched OPSD comparison.

Propagation sensitivity κ . The sensitivity κ controls the sharpness of the propagation gates. Smaller values make the gates less responsive to sequence-relative gaps, while larger values produce sharper changes around the sequence mean. We evaluate $\kappa \in \{1, 2, 5, 10, 20\}$. As shown in Figure 4a, every tested value outperforms the standard OPSD



(a) Fixed values of λ and Inverse-gap.



(b) Coefficient allocation and average scale.

Figure 3: Component ablations of DASH on Qwen3-1.7B. (a) Fixed values of the propagation coefficient λ test adaptive against fixed aggregation, while Inverse-gap reverses the sign of the DASH gate function. (b) Scale-matched OPSD and normalized DASH separate the relative coefficient profile from its average scale. Panel (a) reports the unweighted mean across the three benchmarks. Panel (b) plots improvements over the matched OPSD rerun, with annotations showing absolute Avg@12 scores. All values are means over training seeds 0, 1, 2, 3.

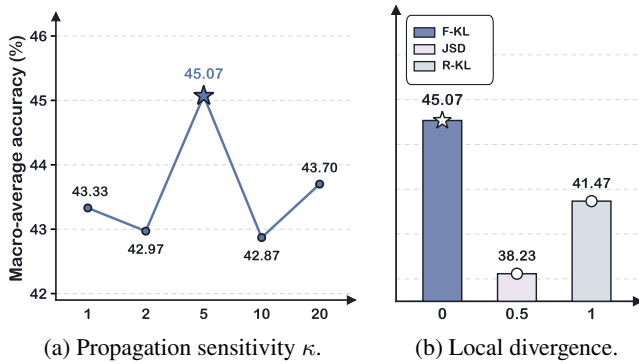


Figure 4: Parameter ablations of DASH on Qwen3-1.7B. (a) Macro-average accuracy under different propagation sensitivities κ . (b) Comparison of forward KL, symmetric JSD, and reverse KL as the local distillation objective. Stars indicate the settings used in the main experiments. Values are means over training seeds 0, 1, 2, 3.

macro-average of 41.87, indicating that the improvement is not limited to a single value of κ . Among the tested settings, $\kappa = 5$ achieves the highest macro-average of 45.07. The lower performance at both smaller and larger values suggests that moderate gate sensitivity better balances insufficient adaptation and overly sharp responses to local gap variations. We therefore use $\kappa = 5$ in the main experiments.

Choice of distillation divergence. We examine the effect of the distillation divergence by replacing the forward KL used in the main configuration with symmetric JSD and reverse KL (Agarwal et al. 2024; Gu et al. 2024; Ko et al. 2024), while keeping all other settings fixed. As shown in Figure 4b, forward KL achieves the highest macro-average

Variant	AIME 2024	AIME 2025	HMMT 2025	Avg.
OPSD, full vocab.	55.60	40.80	29.20	41.87
DASH, full vocab.	58.30	45.80	31.10	45.07
DASH, top-100 + tail	58.10	45.00	30.00	44.37
DASH, top-1 + tail	48.10	33.60	22.80	34.83

Table 2: Effect of vocabulary support on Qwen3-1.7B. The top- k plus tail variants retain the teacher’s top- k tokens and merge the remaining vocabulary into a single tail term. Values are four-seed means.

score of 45.07, outperforming symmetric JSD and reverse KL by 6.84 and 3.60 points, respectively. It also performs best on all three benchmarks. These results show that the divergence choice substantially affects DASH and support the use of forward KL in the main configuration.

Vocabulary support. The main configuration computes the local forward-KL signal over the full vocabulary. We examine whether this distributional support can be compressed by retaining the teacher’s top- k tokens and merging the remaining vocabulary into a single tail term, while keeping all other settings unchanged.

As shown in Table 2, retaining the top 100 teacher tokens yields an average score of 44.37, only 0.70 points below the full-vocabulary configuration and 2.50 points above standard OPSD. In contrast, reducing the explicit support to only the top teacher token lowers the average score to 34.83, a drop of 10.24 points from the main configuration. These results indicate that a moderately compressed support preserves most of the benefit of DASH, whereas an overly concentrated approximation discards important distributional information. We therefore use the full vocabulary in the main experiments.

6 Conclusion

In this work, we revisit how dense token-level supervision should be allocated in on-policy self-distillation. Vanilla OPSD assigns uniform weights to local distillation signals, overlooking the temporal evolution of teacher–student discrepancies along each trajectory. To address this limitation, we propose Divergence-Adaptive Supervision Horizons (DASH), a sequence-aware method for adaptive supervision allocation. DASH transforms sequence-relative divergence gaps into adaptive propagation gates and performs backward multi-step aggregation to construct temporally conditioned token-level weights and adaptive supervision horizons. Experiments across three mathematical reasoning benchmarks and three model scales show that DASH improves the four-seed mean of our matched vanilla OPSD reruns and obtains the highest scores among the compared results. DASH reuses the local teacher and student distributions already computed by vanilla OPSD and requires no additional teacher or student forward passes. We hope our work can offer new insights for improving dense supervision allocation in on-policy learning and long-horizon reasoning.

References

- Agarwal, R.; Vieillard, N.; Zhou, Y.; Stanczyk, P.; Ramos Garea, S.; Geist, M.; and Bachem, O. 2024. On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes. In *International Conference on Learning Representations*.
- Bengio, S.; Vinyals, O.; Jaitly, N.; and Shazeer, N. 2015. Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks. In *Advances in Neural Information Processing Systems*.
- Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; Pinto, H. P. d. O.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*.
- Cho, K.; van Merriënboer, B.; Gulcehre, C.; Bahdanau, D.; Bougares, F.; Schwenk, H.; and Bengio, Y. 2014. Learning Phrase Representations Using RNN Encoder–Decoder for Statistical Machine Translation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*.
- Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; Hesse, C.; and Schulman, J. 2021. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*.
- Gu, Y.; Dong, L.; Wei, F.; and Huang, M. 2024. MiniLLM: Knowledge Distillation of Large Language Models. In *International Conference on Learning Representations*.
- Guha, E.; Marten, R.; Keh, S.; Raoof, N.; Smyrnis, G.; Bansal, H.; Nezhurina, M.; Mercat, J.; Vu, T.; Sprague, Z.; et al. 2025. OpenThoughts: Data Recipes for Reasoning Models. *arXiv preprint arXiv:2506.04178*.
- Guo, D.; Yang, D.; Zhang, H.; et al. 2025. DeepSeek-R1 Incentivizes Reasoning in LLMs through Reinforcement Learning. *Nature*, 645: 633–638.
- Harvard-MIT Mathematics Tournament. 2025. HMMT February 2025: Problems and Solutions. <https://www.hmmt.org/www/archive/282>.
- Hendrycks, D.; Burns, C.; Kadavath, S.; Arora, A.; Basart, S.; Tang, E.; Song, D.; and Steinhardt, J. 2021. Measuring Mathematical Problem Solving with the MATH Dataset. In *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*.
- Hinton, G.; Vinyals, O.; and Dean, J. 2015. Distilling the Knowledge in a Neural Network. *arXiv preprint arXiv:1503.02531*.
- Hochreiter, S.; and Schmidhuber, J. 1997. Long Short-Term Memory. *Neural Computation*, 9(8): 1735–1780.
- Hu, E. J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; and Chen, W. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*.
- Jin, W.; Min, T.; Yang, Y.; Kadhe, S. R.; Zhou, Y.; Wei, D.; Baracaldo, N.; and Lee, K. 2026. Entropy-Aware On-Policy Distillation of Language Models. *arXiv preprint arXiv:2603.07079*.
- Kazemnejad, A.; Aghajohari, M.; Portelance, E.; Sordani, A.; Reddy, S.; Courville, A.; and Le Roux, N. 2024. VinePPO: Unlocking RL Potential for LLM Reasoning through Refined Credit Assignment. *arXiv preprint arXiv:2410.01679*.
- Kim, Y.; and Rush, A. M. 2016. Sequence-Level Knowledge Distillation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*.
- Ko, J.; Kim, S.; Chen, T.; and Yun, S.-Y. 2024. DistiLLM: Towards Streamlined Distillation for Large Language Models. In *International Conference on Machine Learning*.
- Lambert, N.; Morrison, J.; Pyatkin, V.; Huang, S.; Ivison, H.; Brahman, F.; Miranda, L. J. V.; Liu, A.; Dziri, N.; Lyu, S.; Gu, Y.; Malik, S.; Graf, V.; Hwang, J. D.; Yang, J.; Bras, R. L.; Tafjord, O.; Wilhelm, C.; Soldaini, L.; Smith, N. A.; Wang, Y.; Dasigi, P.; and Hajishirzi, H. 2024. TULU 3: Pushing Frontiers in Open Language Model Post-Training. *arXiv preprint arXiv:2411.15124*.
- Li, Y.; Zhang, M.; Shen, D.; and Sun, Y. 2026. PHF: Privileged Hidden Flow for On-Policy Self-Distillation. *arXiv preprint arXiv:2606.29340*.
- Lightman, H.; Kosaraju, V.; Burda, Y.; Edwards, H.; Baker, B.; Lee, T.; Leike, J.; Schulman, J.; Sutskever, I.; and Cobbe, K. 2024. Let’s Verify Step by Step. In *International Conference on Learning Representations*.
- Lin, C.; Chen, K.; and Zhang, W. 2026. ReNIO: Reweighting Negative Trajectory Importance for LLM On-Policy Distillation. *arXiv preprint arXiv:2606.23104*.
- Liu, X.; Wang, X.; Ma, Y.; Zhang, Y.; and Xiao, C. 2026a. When Are Teacher Tokens Reliable? Position-Weighted On-Policy Self-Distillation for Reasoning. *arXiv preprint arXiv:2605.21606*.
- Liu, Y.; Lou, J.; Guan, X.; Ji, Y.; Lin, H.; He, B.; Han, X.; Sun, L.; Yu, X.; and Lu, Y. 2026b. Your Teacher Can’t Help You Here: Combating Supervision Fidelity Decay in On-Policy Distillation. *arXiv preprint arXiv:2605.30833*.

- Lopez-Paz, D.; Bottou, L.; Schölkopf, B.; and Vapnik, V. 2016. Unifying Distillation and Privileged Information. In *International Conference on Learning Representations*.
- Mathematical Association of America. 2024. 2024 American Invitational Mathematics Examination. <https://maa.org/maa-invitational-competitions/>.
- Mathematical Association of America. 2025. 2025 American Invitational Mathematics Examination. <https://maa.org/maa-invitational-competitions/>.
- Nguyen, D.; Xiao, H.; Prasad, A.; Khan, Z.; Das, A.; Zhang, A.; Sahu, S.; Lee, H.; Stengel-Eskin, E.; and Bansal, M. 2026. AVSD: Adaptive-View Self-Distillation by Balancing Consensus and Teacher-Specific Privileged Signals. *arXiv preprint arXiv:2605.20643*.
- Ouyang, L.; Wu, J.; Jiang, X.; Almeida, D.; Wainwright, C.; Mishkin, P.; Zhang, C.; Agarwal, S.; Slama, K.; Ray, A.; Schulman, J.; Hilton, J.; Kelton, F.; Miller, L.; Simens, M.; Askell, A.; Welinder, P.; Christiano, P.; Leike, J.; and Lowe, R. 2022. Training Language Models to Follow Instructions with Human Feedback. In *Advances in Neural Information Processing Systems*.
- Parthasarathi, P.; Reymond, M.; Chen, B.; Cui, Y.; and Chandar, S. 2025. GRPO- λ : Credit Assignment Improves LLM Reasoning. *arXiv preprint arXiv:2510.00194*.
- Pinto, L.; Andrychowicz, M.; Welinder, P.; Zaremba, W.; and Abbeel, P. 2018. Asymmetric Actor Critic for Image-Based Robot Learning. In *Robotics: Science and Systems*.
- Rafailov, R.; Sharma, A.; Mitchell, E.; Ermon, S.; Manning, C. D.; and Finn, C. 2023. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. In *Advances in Neural Information Processing Systems*.
- Ross, S.; Gordon, G. J.; and Bagnell, J. A. 2011. A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning. In *Proceedings of the International Conference on Artificial Intelligence and Statistics*.
- Schulman, J.; Moritz, P.; Levine, S.; Jordan, M.; and Abbeel, P. 2016. High-Dimensional Continuous Control Using Generalized Advantage Estimation. In *International Conference on Learning Representations*.
- Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347*.
- Setlur, A.; Nagpal, C.; Fisch, A.; Geng, X.; Eisenstein, J.; Agarwal, R.; Agarwal, A.; Berant, J.; and Kumar, A. 2025. Rewarding Progress: Scaling Automated Process Verifiers for LLM Reasoning. In *International Conference on Learning Representations*.
- Shao, Z.; Wang, P.; Zhu, Q.; Xu, R.; Song, J.; Bi, X.; Zhang, H.; Zhang, M.; Li, Y. K.; Wu, Y.; and Guo, D. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300*.
- Shen, Z.; Tong, J.; Yan, S.; Shen, C.; Chen, H.; Ye, W.; Hu, X.; Miao, R.; Wang, H.; Zhao, J.; Chen, G.; and Ye, J. 2026. Purified OPSD: On-Policy Self-Distillation Without Losing How to Think. *arXiv preprint arXiv:2607.02234*.
- Sutton, R. S. 1988. Learning to Predict by the Methods of Temporal Differences. *Machine Learning*, 3(1): 9–44.
- Sutton, R. S.; McAllester, D.; Singh, S.; and Mansour, Y. 2000. Policy Gradient Methods for Reinforcement Learning with Function Approximation. In *Advances in Neural Information Processing Systems*.
- Uesato, J.; Kushman, N.; Kumar, R.; Song, F.; Siegel, N.; Wang, L.; Creswell, A.; Irving, G.; and Higgins, I. 2022. Solving Math Word Problems with Process- and Outcome-Based Feedback. *arXiv preprint arXiv:2211.14275*.
- Vapnik, V.; and Vashist, A. 2009. A New Learning Paradigm: Learning Using Privileged Information. *Neural Networks*, 22(5–6): 544–557.
- Wang, P.; Li, L.; Shao, Z.; Xu, R.; Dai, D.; Li, Y.; Chen, D.; Wu, Y.; and Sui, Z. 2024. Math-Shepherd: Verify and Reinforce LLMs Step-by-Step without Human Annotations. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*.
- Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Ichter, B.; Xia, F.; Chi, E.; Le, Q. V.; and Zhou, D. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*.
- Weihs, L.; Jain, U.; Liu, I.-J.; Salvador, J.; Lazebnik, S.; Kembhavi, A.; and Schwing, A. 2021. Bridging the Imitation Gap by Adaptive Insubordination. In *Advances in Neural Information Processing Systems*.
- Williams, R. J. 1992. Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. *Machine Learning*, 8(3–4): 229–256.
- Xie, Y.; Zhu, S.; Wen, T.; Chen, B.; and Wang, Y. 2026. On the Position Bias of On-Policy Distillation. *arXiv preprint arXiv:2606.22600*.
- Yang, A.; Li, A.; Yang, B.; Zhang, B.; Hui, B.; Zheng, B.; Yu, B.; Gao, C.; Huang, C.; Lv, C.; et al. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388*.
- Yu, Q.; Zhang, Z.; Zhu, R.; Yuan, Y.; Zuo, X.; Yue, Y.; Fan, T.; Liu, G.; Liu, L.; Liu, X.; Lin, H.; Lin, Z.; Ma, B.; Sheng, G.; Tong, Y.; Zhang, C.; Zhang, M.; Zhang, W.; Zhu, H.; Zhang, J.; Wang, Y.; Wu, Y.; Wang, Y.; et al. 2025. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. *arXiv preprint arXiv:2503.14476*.
- Yue, Y.; Chen, Z.; Lu, R.; Zhao, A.; Wang, Z.; Yue, Y.; Song, S.; and Huang, G. 2025. Does Reinforcement Learning Really Incentivize Reasoning Capacity in LLMs Beyond the Base Model? *arXiv preprint arXiv:2504.13837*.
- Zelikman, E.; Wu, Y.; Mu, J.; and Goodman, N. D. 2022. STaR: Bootstrapping Reasoning with Reasoning. In *Advances in Neural Information Processing Systems*.
- Zhao, S.; Xie, Z.; Liu, M.; Huang, J.; Pang, G.; Chen, F.; and Grover, A. 2026. Self-Distilled Reasoner: On-Policy Self-Distillation for Large Language Models. *arXiv preprint arXiv:2601.18734*.

Overview. This appendix is organized into five parts. Appendix A provides supplementary theoretical analysis of DASH, including the fixed-horizon gradient decomposition, the clipped local signal, the aggregation coefficients, the effective supervision horizon, and the implemented gradient. Appendix B records result provenance, baseline instantiations, the exact checkpoint-selection rule, and implementation details not included in the main paper, including the complete LoRA configuration, optimizer and distributed-training settings, teacher prompts, answer verification, and hardware. Appendix C reports the four-seed OPSD and DASH protocol and uncertainty estimates over random seeds 0, 1, 2, 3. Appendix D presents the complete component and design-choice ablations, including fixed propagation coefficients, gate direction and signals, coefficient scale, propagation sensitivity, divergence geometry, and vocabulary support. Appendix E examines the compatibility of DASH with outcome-level optimization through the complete GRPO grid and a cross-scale extension. Together, these materials complement the theoretical and empirical results in the main text with complete derivations, reproducibility details, and additional experimental evidence.

A Supplementary Theoretical Analysis

This section expands the theoretical analysis in the main text. It first restates and proves the fixed-horizon gradient decomposition, then connects that result to the local signal, aggregation coefficients, effective supervision horizons, and gradients used by DASH.

A.1 Fixed-Horizon Construction

Let π_θ denote the student policy. For a problem x with privileged information z , define the filtration $\mathcal{F}_u = \sigma(x, z, y_{<u})$. Training rollouts use a deterministic maximum completion length (1,024 tokens in the main configuration). We therefore fix a truncation horizon $H \geq 1$ and represent each rollout as a padded sequence $\mathbf{y} = (y_1, \dots, y_H)$. If an end-of-sequence token is emitted at position $T \leq H$, the process subsequently remains in a parameter-independent absorbing state:

$$\pi_\theta(\perp | s_u) = 1, \quad \nabla_\theta \log \pi_\theta(\perp | s_u) = 0, \quad u > T, \quad (13)$$

where $s_u = (x, y_{<u})$. We set every local signal to zero after termination. Conditional on x , the padded trajectory distribution is

$$P_\theta(\mathbf{y} | x) = \prod_{u=1}^H \pi_\theta(y_u | s_u). \quad (14)$$

The main analysis considers the fixed-horizon surrogate

$$\mathcal{J}_H(\theta) = \mathbb{E}_{(x,z) \sim \mathcal{D}} \mathbb{E}_{\mathbf{y} \sim P_\theta(\cdot | x)} \left[\frac{1}{H} \sum_{t=1}^H d_t \right]. \quad (15)$$

For fixed (x, z) , the padded trajectory space is finite, so the gradient and the inner trajectory expectation commute term by term. Moreover, $1/H$ is deterministic and can be moved outside that expectation. These are the two properties needed below; they would not follow from treating the realized length T as a constant outside an expectation over variable-length trajectories.

A.2 Privileged Teacher

Adapter-disabled privileged teacher. The privileged teacher is the frozen base network evaluated with the LoRA adapter disabled. Its parameters $\bar{\theta}$ therefore remain independent of the trainable adapter parameters θ . Thus,

$$\pi_t^T = \text{sg}[\pi_{\bar{\theta}}(\cdot | s_t, z)], \quad \nabla_\theta \pi_t^T = 0. \quad (16)$$

Under this frozen-teacher setting, the local forward KL

$$d_t = D_{\text{KL}}(\pi_t^T \| \pi_\theta(\cdot | s_t)) \quad (17)$$

depends explicitly on θ only through the student conditional. Accordingly, $\nabla_\theta^{\text{loc}} d_t$ denotes differentiation through that conditional at the fixed sampled prefix, with both the teacher target and the discrete sampling path detached.

A.3 Decomposition and Proof

Proposition 2 (Fixed-Horizon Gradient Decomposition). *Let ρ_t be an $\mathcal{F}_t$ -measurable per-position scalar whose explicit dependence on θ at a fixed sampled prefix is differentiable almost surely, and set $\rho_t = 0$ for $t > T$. Define*

$$\mathcal{J}_H^\rho(\theta) = \mathbb{E} \left[\frac{1}{H} \sum_{t=1}^H \rho_t \right]. \quad (18)$$

Then

$$\begin{aligned} \nabla_\theta \mathcal{J}_H^\rho(\theta) = \mathbb{E} \left[\frac{1}{H} \sum_{t=1}^H \nabla_\theta^{\text{loc}} \rho_t \right. \\ \left. + \frac{1}{H} \sum_{u=1}^H G_{u+1}^\rho \nabla_\theta \log \pi_\theta(y_u | s_u) \right], \end{aligned} \quad (19)$$

where $G_{u+1}^\rho := \sum_{t=u+1}^H \rho_t$ is the future signal-to-go after position u .

Proof. Let $R(\mathbf{y}; \theta) = \sum_{t=1}^H \rho_t$. The finite-horizon construction and the log-derivative identity give

$$\begin{aligned} \nabla_\theta \mathcal{J}_H^\rho(\theta) = \frac{1}{H} \mathbb{E} \left[\sum_{t=1}^H \nabla_\theta^{\text{loc}} \rho_t \right. \\ \left. + R(\mathbf{y}; \theta) \nabla_\theta \log P_\theta(\mathbf{y} | x) \right]. \end{aligned} \quad (20)$$

By Eq. (14),

$$\nabla_\theta \log P_\theta(\mathbf{y} | x) = \sum_{u=1}^H \nabla_\theta \log \pi_\theta(y_u | s_u). \quad (21)$$

The trajectory-dependent component is therefore

$$\sum_{u=1}^H \sum_{t=1}^H \rho_t \nabla_\theta \log \pi_\theta(y_u | s_u). \quad (22)$$

Fix $t \leq u$. Because ρ_t is $\mathcal{F}_t$ -measurable and $\mathcal{F}_t \subseteq \mathcal{F}_u$, it can be taken outside the conditional expectation over y_u . The score has zero conditional mean even after conditioning

on the privileged information z , since y_u is sampled from $\pi_\theta(\cdot | s_u)$. Hence, by the tower property,

$$\begin{aligned} & \mathbb{E}[\rho_t \nabla_\theta \log \pi_\theta(y_u | s_u)] \\ &= \mathbb{E}[\rho_t \mathbb{E}[\nabla_\theta \log \pi_\theta(y_u | s_u) | \mathcal{F}_u]] = 0. \end{aligned} \quad (23)$$

Only terms with $t > u$ remain in Eq. (22); their inner sum is G_{u+1}^ρ . Substitution into Eq. (20) proves Eq. (19). $\square$

Specialization and padded positions. Taking $\rho_t = d_t$ recovers Proposition 1. The causal cancellation above also applies to the clipped signal r_t defined below because it is determined by (s_t, z) before y_t is sampled. For padded positions, both the local signal and the score are zero by construction, so the result is unaffected by the amount of padding between T and H .

A.4 Clipped Local Signal

Let

$$\ell_{t,v} = \pi_t^T(v) \log \frac{\pi_t^T(v)}{\pi_t^S(v)} \quad (24)$$

be the vocabulary-level summand of the forward KL, so that $d_t = \sum_{v \in \mathcal{A}} \ell_{t,v}$. DASH applies the same pointwise upper clipping used in the main text:

$$r_t = \sum_{v \in \mathcal{A}} \min(\ell_{t,v}, \tau). \quad (25)$$

Thus, $r_t \leq d_t$ and $r_t \leq \tau|\mathcal{A}|$. Because individual forward-KL summands can be negative, the upper-clipped sum is not guaranteed to be non-negative. This does not affect its use as the centered gate signal or as a local differentiable loss: r_t is $\mathcal{F}_t$ -measurable, and the usual automatic-differentiation subgradient is used at the clipping threshold.

A.5 Aggregation Coefficients and Supervision Horizons

The DASH recursion

$$A_T = r_T, \quad A_t = r_t + \lambda_t A_{t+1} \quad (26)$$

unrolls as

$$A_t = \sum_{k=t}^T \left(\prod_{j=t}^{k-1} \lambda_j \right) r_k, \quad (27)$$

where an empty product equals one. Summing over all starting positions and exchanging the order of summation gives

$$\mathcal{L}_{\text{DASH}} = \frac{1}{T} \sum_{k=1}^T c_k r_k, \quad c_k = \sum_{t=1}^k \prod_{j=t}^{k-1} \lambda_j. \quad (28)$$

Separating the $t = k$ term yields the recursion stated in the main paper:

$$c_1 = 1, \quad c_k = 1 + \lambda_{k-1} c_{k-1}. \quad (29)$$

Since each sigmoid gate satisfies $0 < \lambda_j < 1$,

$$1 \leq c_k \leq k. \quad (30)$$

For a constant gate λ , the exact finite-length expression is

$$c_k = \frac{1 - \lambda^k}{1 - \lambda}. \quad (31)$$

The effective supervision horizon beginning at position t is the total propagation mass assigned to the current and later positions:

$$h_t = \sum_{k=t}^T \prod_{j=t}^{k-1} \lambda_j. \quad (32)$$

For a constant gate,

$$h_t = \frac{1 - \lambda^{T-t+1}}{1 - \lambda}, \quad (33)$$

which approaches $1/(1 - \lambda)$ as the remaining rollout length grows. Under DASH, $\lambda_t = \text{sg}[\sigma(-\kappa(r_t - \bar{r}))]$, so a below-average signal leaves the boundary after position t more open, whereas an above-average signal restricts propagation across that boundary. The coefficient and horizon views satisfy

$$\sum_{k=1}^T c_k = \sum_{t=1}^T h_t, \quad (34)$$

because both sides sum the same gate products over all pairs $1 \leq t \leq k \leq T$.

A.6 Gradient of the Implemented DASH Objective

The gate construction is detached:

$$\lambda_t = \text{sg}[\sigma(-\kappa(r_t - \bar{r}))], \quad \bar{r} = \frac{1}{T} \sum_{t=1}^T r_t. \quad (35)$$

The sequence mean and all subsequent operations used to construct λ_t and c_k therefore carry no gradient. Conditional on a sampled rollout, the implemented backward pass is

$$\nabla_\theta \mathcal{L}_{\text{DASH}} = \frac{1}{T} \sum_{k=1}^T c_k \nabla_\theta^{\text{loc}} r_k. \quad (36)$$

Thus, DASH performs sequence-conditioned coefficient allocation over direct local-distillation gradients. The mean $\bar{r}$ couples positions when the detached coefficients are constructed, but it does not introduce cross-position gradients through the gates.

A.7 Relation to the Implemented Length Normalization

Proposition 2 deliberately concerns the fixed-horizon surrogate $\mathcal{J}_H^\rho$, whereas the implemented losses normalize each sampled rollout by its realized length T . For a fixed rollout, replacing $1/H$ with $1/T$ rescales all local coefficients by the same positive scalar. It therefore preserves both the uniform within-rollout coefficient profile of vanilla OPSD and the relative DASH profile $(c_1, \dots, c_T)$. We do not claim that the gradient of the variable-length expected objective is identical to Eq. (19).

A.8 Scope of the Decomposition

The second term in Eq. (19) is a score-function trajectory term weighted by a sum of subsequent local signals. DASH

does not estimate this term, introduce score-function gradients, or perform future-to-past credit assignment. The coefficient families are also structurally different: G_{u+1}^p aggregates signals after position u and multiplies the score term, whereas c_k is determined by the preceding gate path and multiplies the direct local-distillation gradient at position k . The decomposition is used only to motivate order-dependent coefficient allocation within the differentiable local distillation channel; it does not imply that DASH recovers the omitted trajectory gradient.

B Additional Reproducibility Details

This section makes explicit which results are externally reported, which are produced by our reruns, how each comparison method is instantiated, and how checkpoints are selected. It then records the remaining implementation details recovered from the saved run artifacts and launch scripts.

B.1 Result Provenance and Comparison Scope

Table 3 separates external reference results from our reruns. Base, SFT, and GRPO are taken from Zhao et al. (Zhao et al. 2026) and are marked with † in Table 1. These external values are not subjected to our 200-step budget or checkpoint-selection rule. All remaining entries in Table 1 are produced by us.

Method	Source	Training runs
Base	Zhao et al.	Not applicable
SFT, GRPO	Zhao et al.	As reported
OPSD	Our rerun	$s \in \{0, 1, 2, 3\}$
EOPD	Our rerun	One ($s = 0$)
AVSD	Our rerun	One ($s = 0$)
PW-OPSD	Our rerun	One ($s = 0$)
DASH	Our rerun	$s \in \{0, 1, 2, 3\}$
DASH ablations	Our reruns	$s \in \{0, 1, 2, 3\}$

Table 3: Sources and training repetitions of the compared results.

The four-seed OPSD and DASH means reduce dependence on any single training initialization. They are not interpreted as statistical significance tests against EOPD, AVSD, or PW-OPSD, which are single-run reruns, or against the external Base, SFT, and GRPO results. Accordingly, claims involving these methods refer to the compared or displayed results rather than to statistically significant differences.

B.2 Baseline Instantiations

For OPSD, EOPD, PW-OPSD, and DASH, the student receives only the problem and generates its own on-policy rollout. The privileged teacher uses the same base network with the LoRA adapter disabled, additionally conditions on the reference solution (including the final answer), and scores the student-generated tokens by teacher forcing on that rollout. These methods share the same base checkpoint, data preprocessing, student rollout configuration, and privileged-teacher context in our reruns, while retaining their method-specific objectives.

OPSD uniformly averages the local forward-KL signals. EOPD instantiates its entropy-aware objective on this matched privileged-OPSD backbone; it therefore isolates the transfer of the EOPD objective rather than reproducing the external-teacher architecture used in the original paper. PW-OPSD retains its predefined position-dependent weights. DASH replaces uniform or predefined weighting with the proposed sequence-conditioned multi-step aggregation.

AVSD retains its method-defining multi-view teacher construction rather than using the single reference-conditioned teacher above. Its teacher views are constructed from the full solution, a partial solution, and the final answer. Apart from this method-specific construction and objective, our AVSD run uses the same base model, training data, 200-step outer budget, checkpoint-selection rule, and final evaluation procedure as the other reruns.

B.3 Training Budget and Checkpoint Selection

Every method rerun by us is trained for the complete 200 optimization steps. We save checkpoints every 20 steps, giving the candidate set

$$\mathcal{C} = \{20, 40, \dots, 200\}. \quad (37)$$

Fix a method and model scale. For training seed s , candidate checkpoint $t \in \mathcal{C}$, and benchmark b , let $m_{s,t,b}$ denote the corresponding Avg@12 score. We define the selection score

$$a_{s,t} = \frac{1}{3} \sum_{b \in \mathcal{B}} m_{s,t,b}, \quad \mathcal{B} = \{\text{AIME24, AIME25, HMMT25}\}, \quad (38)$$

and select

$$t_s^* = \arg \max_{t \in \mathcal{C}} a_{s,t}. \quad (39)$$

All three benchmark scores for seed s come from this single selected checkpoint:

$$m_{s,b} = m_{s,t_s^*,b}. \quad (40)$$

For a single-run rerun, $s = 0$ and the selected scores are reported directly without averaging across training runs. Because $\mathcal{B}$ is also the final benchmark set, this is a best-within-200-step reporting protocol, not held-out validation selection.

Adapters and optimization. For the matched privileged-OPSD runs, the adapter configuration is identical at all three model scales. LoRA is applied to the attention projections `q_proj`, `k_proj`, `v_proj`, and `o_proj`, and to the MLP projections `gate_proj`, `up_proj`, and `down_proj`. The effective scaling is $\alpha/r = 2.0$ and the adapter dropout is 0.05. For causal language modeling, bias, rank-stabilized LoRA, DoRA, and additional trainable modules are disabled. The A matrix uses Kaiming initialization and B is initialized to zero. The base model, embeddings, and language-model head remain frozen.

We use fused AdamW with $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, and zero weight decay. The learning rate decays linearly from the value stated in the main text to zero over the fixed optimization horizon, without warmup. Although the trainer stores `num_train_epochs=30`, the step limit is

reached first and therefore determines the training duration. Training uses bfloat16 mixed precision, DeepSpeed ZeRO-2 with optimizer-state offloading, gradient checkpointing, and FlashAttention-2. The maximum combined sequence length is 20,000 tokens. The privileged-teacher forward pass is evaluated with the adapter disabled, so only the student LoRA path receives parameter updates.

Batch decomposition and runtime. Table 4 combines the scale-specific DASH batch decomposition and runtime information that previously required two separate tables. All three configurations realize the shared effective batch size of 64.

Model	Micro/accum./GPU	Hardware; time
Qwen3-1.7B	4/2/8	8× A800; ~45 min
Qwen3-4B	2/4/8	8× A800; ~2.3 h
Qwen3-8B	1/16/4	4× A800; ~15 h

Table 4: DASH batch decomposition and approximate runtime. The middle column reports per-device batch size, gradient accumulation, and GPU count; runtime includes scheduler and queue overhead.

All DASH runs use NVIDIA A800-SXM4-80GB GPUs. On-policy rollout generation dominates runtime. Relative to vanilla OPSD, DASH requires no additional teacher or student forward pass; its extra scalar backward scan accounts for less than 1% of the step time in matched run timestamps.

Matched OPSD-family prompts. For OPSD, EOPD, PW-OPSD, and DASH, the student receives only the problem, rendered with the Qwen3 chat template using `enable_thinking=True` and `add_generation_prompt=True`; no additional instruction is appended. It then generates its own on-policy rollout. The privileged teacher uses the same base network with the LoRA adapter disabled. It receives the problem and reference solution, including the final answer, followed by the two instruction strings below, and scores the student-generated tokens by teacher forcing on the same rollout.

The reference reasoning above arrives at the correct answer. Please analyze this solution and explain the key reasoning steps and problem-solving strategies employed. Do NOT use <think> tags. Do NOT derive your own solution. Simply analyze and explain the reference solution provided above .

After reading the reference solution above, make sure you truly understand the reasoning behind each step -- do not copy or paraphrase it. Now, using your own words and independent reasoning, derive the same final answer to the problem above. Think step by step, explore different

approaches, and don't be afraid to backtrack or reconsider if something doesn't work out:

In these matched OPSD-family runs, the privileged reference is never provided to the student during rollout generation or benchmark evaluation. Training sequences use right padding.

Answer extraction and equivalence checking. The reported benchmark scores use a symbolic verifier. For each model response, the evaluator extracts the last occurrence of `\boxed{...}`. If the extracted prediction contains no math delimiters, it is wrapped in dollar signs. The prediction and ground truth are parsed with `math_verify.parse(..., fallback_mode="no_fallback")`, and equivalence is checked with a five-second timeout using `math_verify.verify`. If parsing or verification raises an exception, the evaluator falls back to exact comparison after removing dollar signs and spaces and converting both strings to lowercase.

The optional outcome-level extension uses a separate reward-time grader. It extracts the last boxed answer, applies the EleutherAI-style `strip_string` normalization, and checks exact string equality. The normalization canonicalizes common forms involving fractions, square roots, delimiters, units, percentages, degree symbols, spaces, and simple decimal or division expressions. This grader is used only by the optional outcome-level training objective, not for any reported benchmark score.

C Four-Seed Results and Statistical Robustness

The matched OPSD configurations, the main DASH configurations, and all DASH ablations are trained with four random seeds, $s \in \{0, 1, 2, 3\}$. For benchmark b , let $m_{s,b}$ denote the Avg@12 score from the checkpoint selected for seed s by Eq. (39). We report the arithmetic mean and sample standard deviation

$$\mu_b = \frac{1}{4} \sum_{s=0}^3 m_{s,b}, \quad \text{Std}_b = \sqrt{\frac{1}{3} \sum_{s=0}^3 (m_{s,b} - \mu_b)^2}. \quad (41)$$

The four training seeds are distinct from the 12 independently sampled responses used to compute Avg@12 for each problem.

The OPSD and DASH entries in Table 1 are four-seed means. In particular, the Qwen3-1.7B OPSD mean (55.60, 40.80, 29.20), with macro-average 41.87, is our matched rerun and serves as the common OPSD anchor in all Qwen3-1.7B ablations. It is not the OPSD value reported by Zhao et al. (Zhao et al. 2026).

The benchmark-level standard deviations range from 0.30 to 1.30 points across the three model scales. The means in Table 5 are those reported in the main paper. We do not derive a standard deviation for the macro-average from the three marginal standard deviations because the benchmark

Benchmark	1.7B	4B	8B
AIME 2024	58.30 $\pm$ 0.90	77.20 $\pm$ 0.70	78.90 $\pm$ 0.90
AIME 2025	45.80 $\pm$ 1.20	71.10 $\pm$ 0.80	71.40 $\pm$ 1.00
HMMT 2025	31.10 $\pm$ 0.30	46.70 $\pm$ 0.70	48.90 $\pm$ 1.30
Macro avg.	45.07	65.00	66.40

Table 5: Four-seed DASH results. Benchmark rows report mean $\pm$ sample standard deviation over seeds 0, 1, 2, 3 ($n = 4$); the last row is the arithmetic mean of the three benchmark means.

scores are paired within each training seed. The four-seed means reduce dependence on any single training initialization; they are not used to claim statistical significance against the single-run or externally reported baselines.

D Complete Ablation Results

This section expands the ablations summarized in the main text with complete benchmark-level results and the additional gate-signal study. Unless otherwise specified, all variants use Qwen3-1.7B and the same training, checkpoint-selection, and evaluation protocol. Each entry, including the OPSD configuration obtained with $\lambda = 0$, is the mean Avg@12 score over training seeds 0, 1, 2, 3; the final column is the unweighted mean of the three benchmark scores. Thus, the OPSD reference of 41.87 is our matched four-seed rerun rather than an external result. A24, A25, and H25 abbreviate AIME 2024, AIME 2025, and HMMT February 2025.

Propagation coefficient and gate direction. To separate multi-step aggregation from sequence-adaptive propagation, we replace the token-wise gates with a constant λ shared by all positions and rollouts; $\lambda = 0$ recovers vanilla OPSD. The inverse-gap control instead preserves the sequence-relative gap, sigmoid gate, sensitivity, and backward recursion but reverses the sign of the gate input:

$$\lambda_t^{\text{Inv}} = \text{sg}[\sigma(+\kappa(r_t - \bar{r}))]. \quad (42)$$

Setting	A24	A25	H25	Avg.
OPSD ($\lambda = 0$)	55.60	40.80	29.20	41.87
Fixed $\lambda = 0.1$	58.10	44.20	28.60	43.63
Fixed $\lambda = 0.3$	58.10	42.20	29.40	43.23
Fixed $\lambda = 0.5$	55.60	42.20	30.30	42.70
Fixed $\lambda = 0.7$	56.40	41.90	29.40	42.57
Fixed $\lambda = 0.9$	56.70	41.90	28.90	42.50
Inverse-gap	56.90	41.90	27.50	42.10
DASH	58.30	45.80	31.10	45.07

Table 6: Four-seed means for fixed-coefficient and gate-direction controls; $\lambda = 0$ is the matched OPSD reference.

Every fixed coefficient improves over OPSD in macro-average; $\lambda = 0.1$ is strongest among them at 43.63. Inverse-gap reaches 42.10, whereas DASH reaches 45.07, supporting sequence-conditioned propagation and the selected gap-to-gate direction.

Coefficient allocation, average scale, and gate signal. Table 7 separates relative coefficient allocation from average scale and then varies the propagation-gate signal. Soft-OR smoothly combines the entropy and divergence-gap gates; the OPSD and DASH rows in the first panel serve as shared anchors.

Variant	A24	A25	H25	Avg.
<i>Coefficient allocation and scale</i>				
OPSD	55.60	40.80	29.20	41.87
Scale-matched OPSD	56.40	41.90	29.40	42.57
Normalized DASH	57.80	44.40	30.60	44.27
DASH	58.30	45.80	31.10	45.07
<i>Propagation-gate signal</i>				
Entropy ($\kappa = 1$)	58.60	45.60	28.90	44.37
Entropy ($\kappa = 2$)	55.80	42.50	30.30	42.87
Entropy ($\kappa = 5$)	54.70	43.90	27.50	42.03
Soft-OR ($\kappa = 5$)	56.90	41.90	32.50	43.77

Table 7: Four-seed means for coefficient-allocation controls and alternative gate signals. OPSD and DASH are the shared anchors.

Dynamic allocation improves macro-average by 2.40 points at the OPSD scale and 2.50 at the DASH scale; increasing scale alone contributes 0.70 and 0.80 points. Entropy and Soft-OR gates also improve over OPSD for suitable settings, while the divergence-gap anchor is strongest. Entropy changes from 44.37 at $\kappa = 1$ to 42.03 at $\kappa = 5$.

Sensitivity, divergence geometry, and vocabulary support. Table 8 collects the remaining choices. The OPSD and DASH rows are shared anchors; tail rows merge teacher mass outside the retained top- k vocabulary.

Setting	A24	A25	H25	Avg.
<i>Shared anchors</i>				
OPSD	55.60	40.80	29.20	41.87
DASH	58.30	45.80	31.10	45.07
<i>Propagation sensitivity (alternatives to $\kappa = 5$)</i>				
$\kappa = 1$	58.60	42.20	29.20	43.33
$\kappa = 2$	57.50	43.10	28.30	42.97
$\kappa = 10$	55.80	43.60	29.20	42.87
$\kappa = 20$	58.60	43.10	29.40	43.70
<i>Local divergence (alternatives to forward KL)</i>				
Symmetric JSD	51.90	38.10	24.70	38.23
Reverse KL	54.70	41.10	28.60	41.47
<i>Vocabulary support (alternatives to full vocabulary)</i>				
Top-100 + tail	58.10	45.00	30.00	44.37
Top-1 + tail	48.10	33.60	22.80	34.83

Table 8: Four-seed means for propagation sensitivity, divergence, and vocabulary support. DASH denotes $\kappa = 5$, forward KL, and full vocabulary.

All sensitivities exceed OPSD, with $\kappa = 5$ strongest at 45.07. Forward KL exceeds symmetric JSD and reverse KL by 6.84 and 3.60 macro-average points. Top-100 support is

only 0.70 points below the full vocabulary and remains 2.50 above OPSD; top-1 support falls to 34.83.

E Compatibility with Outcome-Level RL

We optionally add a GRPO correctness term (weight η , K rollouts) to DASH; $\gamma = \lambda = 0.95$ aggregation affects only the GRPO path. Results are four-seed Avg@12 means. Aggregation improves all four matched 1.7B settings (Table 9). The strongest hybrid, $\eta = 0.3, K = 8$, obtains (57.80, 45.00, 31.70; 44.83) on A24, A25, H25, and average, versus (58.30, 45.80, 31.10; 45.07) for DASH. At 8B, the same hybrid reaches (80.00, 73.30, 48.60; 67.30) versus DASH’s (78.90, 71.40, 48.90; 66.40), so compatibility is scale-dependent.

η	K	No agg.	Agg.	Gain
0.3	4	42.80	43.17	+0.37
0.3	8	43.13	44.83	+1.70
0.5	4	42.50	42.97	+0.47
0.5	8	42.50	44.50	+2.00

Table 9: Four-seed Qwen3-1.7B macro-averages for GRPO-path aggregation.